AF 52 - A

Capstone Project Report
On
वज्रSense
AI/ML-Based Short-Term Convective Activity Nowcasting Prototype for Mumbai

Submitted in partial fulfillment of the requirements

*of the **Third Year** of*

Bachelor of Technology
In
Computer Engineering

by

Ishita Patil [EU1242058]

Arya Patil [EU1242117]

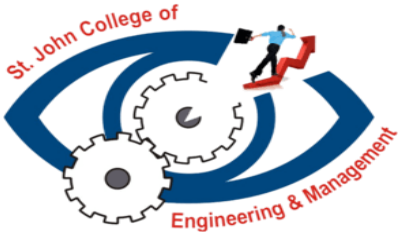
Onkar Pawar [EU1242088]

Pankaj Rana [EU1242077]

Under the Guidance of

Ms. Deepika Nadar

Astt. Professor, Computer Department., SJCEM



DEPARTMENT OF COMPUTER ENGINEERING
ST. JOHN COLLEGE OF ENGINEERING & MANAGEMENT
UNIVERSITY OF MUMBAI

2025-2026

St. John College of Engineering and Management
Autonomous Institute
(A Christian Religious Minority Institution)
Approved by AICTE and DTE, Affiliated to University of Mumbai / MSBTE
DTE Code : 3218 AICTE Permanent ID : 1-4790201
NAAC Accredited with Grade 'A+', Three Programs NBA Accredited

Certificate of Approval

This is to certify that, following are confide students of T. Tech in Computer Engineering Department. They have satisfactorily completed the requirements of Capstone Project for the courses as prescribed by **ST. JOHN COLLEGE OF ENGINEERING AND MANAGEMENT (An Autonomous College affiliated to University of Mumbai),** while working on **वज्रSense**

AI/ML-Based Short-Term Convective Activity Nowcasting

Ishita Patil [EU1242058]

Arya Patil [EU1242117]

Onkar Pawar [EU1242088]

Pankaj Rana [EU1242077]

Signature: _____ **Name: Mrs. Deepika Nadar** *Assistant Professor, Computer Engineering, SJCEM*	Signature: _____ **Name: Dr. Sunny Sall** *HOD, Computer Engineering, SJCEM*
Signature: _____ **Name: Dr. Kamal Shah** *Principal, SJCEM*	Signature: _____ **Name: External Examiner** *External Examiner Designation, Industry / University Nominee*

Declaration

I/we declare that this written submission represents my/our ideas in my/our own words and where others' ideas or words have been included, I/we have adequately cited and referenced the original sources. I/we also declare that I/we have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in my/our submission. I/we understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

Signatures: _____

Ishita Patil [EU1242058]

Arya Patil [EU1242117]

Onkar Pawar [EU1242088]

Pankaj Rana [EU1242077]

Date:

ACKNOWLEDGEMENT

We sincerely thank to our guide Ms. Deepika Nadar for her guidance and support for carrying out our project work.

Ishita Patil [EU1242058]

Arya Patil [EU1242117]

Onkar Pawar [EU1242088]

Pankaj Rana [EU1242077]

ABSTRACT

Thunderstorms and lightning are important atmospheric hazards that can affect public safety, transportation, electrical infrastructure, agriculture, and outdoor activities. The rapid development of convective weather systems makes short-term weather analysis an important area of meteorological research. Artificial Intelligence (AI) and Machine Learning (ML) provide techniques for analysing weather-related variables and identifying patterns that may be associated with convective conditions. VajraSense is an AI/ML-based prototype developed to estimate short-term convective weather risk using meteorological variables and an interactive visualization system. The project focuses on integrating weather data processing, feature engineering, machine learning, fuzzy logic, and a web-based dashboard into a single platform.

The system considers variables such as temperature, relative humidity, dew point, precipitation, wind speed, wind gusts, atmospheric pressure, cloud cover, Convective Available Potential Energy (CAPE), Lifted Index (LI), and Convective Inhibition (CIN). These variables are processed to prepare the input features required by the model. The dashboard presents the resulting estimates through risk cards, a geographical map, atmospheric telemetry, short-term outlook panels, and an interactive What-If Simulator. The fuzzy-logic simulator allows users to modify selected atmospheric inputs and observe how the configured fuzzy system responds to hypothetical conditions. This provides an educational and exploratory method for understanding relationships between selected inputs and estimated risk.

The prototype demonstrates the integration of data processing, machine learning, fuzzy inference, and interactive visualization. However, the currently described target is partly rule-derived, and independent observed lightning data have not been established for validation. Therefore, the system is an experimental decision-support prototype rather than a verified operational lightning-warning service. Future work will focus on independent lightning observations, improved model evaluation, data-source integration, uncertainty estimation, and operational reliability.

Keywords: Artificial Intelligence, Machine Learning, Thunderstorm Nowcasting, Lightning Risk, Meteorological Data, Fuzzy Logic, Weather Visualization.

CONT EN TS

- List of Figures i
- List of Tables ii
- Abbreviations and symbols iii

Chapter No.	Topic	Page No.
	Overview	**1**
	1.1Introduction	1
	1.2 Background	2
Chapter1	1.3 Importance of the project	2
	1.4 Perspective of stakeholders and customers	3
	1.5Objectives and scope of the project	4
	1.6 Summary	5
	Literature Survey & Proposed work	**6**
	2.1Introduction	7
	2.2 Literature Survey Table	7
Chapter2	2.3Problem Definition	8
	2.4Feasibility Study	8
	2.5Methodology Used	9
	2.6 Summary	10
	Analysis and Planning	**11**
	3.1Introduction	12
Chapter3	3.2Project Planning	13
	3.3 Scheduling	14
	3.4 Summary	15
	Design & Implementation	**16**
	4.1 DFD	16
	4.2Block Diagram	17
Chapter4	4.3FlowChart	18
	4.4 UML	19
	4.5 GUI Screenshot	20
	Results & Discussion	**21**
	5.1Actual Results	22
Chapter5	5.2Future Scope	23
	5.3 Testing	24
	5.4 Deployment	25
Chapter6	**Conclusion**	**26**
	References	**27**

List of Figures

Figure No.	Figure Name	Page No.
1.	Example of convective cloud development associated with thunderstorms.	1
2.	Data Flow Diagram (DFD) — Level 0	14
3.	Data Flow Diagram (DFD) — Level 1	15
4.	Block Diagram of the VajraSense System	17
5.	Flowchart of the Proposed System	18
6.	UML Use Case Diagram	21
7.	UML Activity Diagram	22
8.	UML Component Diagram	22
9.	VajraSense Landing Page (GUI 01)	23
10.	Main Dashboard Interface (GUI 02)	24
11.	Risk Estimation and Explainability Interface (GUI 03)	25
12.	Fuzzy What-If Simulator (GUI 04)	26
13.	Deployment Architecture of VajraSense	27

CHAPTER 1: OVERVIEW

1.1 Introduction

Weather is a dynamic natural process influenced by atmospheric temperature, humidity, pressure, wind, cloud formation, and solar heating. Under suitable atmospheric conditions, rising air can develop into convective clouds and thunderstorms. These systems may produce lightning, heavy rainfall, strong winds, and other hazardous weather conditions.

Thunderstorms can develop and change rapidly. Understanding the atmospheric conditions associated with convection is therefore important for short-term weather analysis. Meteorological observations, weather models, radar systems, and satellite imagery provide information about atmospheric conditions, but interpreting multiple variables together may be challenging.

Artificial Intelligence and Machine Learning have become useful approaches for analysing complex datasets. Machine-learning algorithms can learn relationships between input variables and predefined targets. When applied carefully to meteorological data, these techniques can support the analysis of atmospheric patterns.

VajraSense is designed as a prototype that combines weather-related data processing with machine-learning-based risk estimation and an interactive dashboard. It aims to organize selected atmospheric variables and present the system's estimates in a form that is easier to inspect and understand.

The project also incorporates a fuzzy-logic What-If Simulator. This feature allows users to change selected inputs and examine the corresponding output of a rule-based fuzzy system. The simulator helps users explore hypothetical scenarios without requiring manual recalculation of the configured rules.

The main focus of VajraSense is the integration of data processing, ML-based estimation, fuzzy logic, and visualization. The project also identifies the importance of data quality, independent validation, and clear communication of model limitations.



Figure 1. Example of convective cloud development associated with thunderstorms.

1.1.1 Need for the Project

Meteorological information may be available through different datasets and interfaces. Users may need to inspect several variables before forming an understanding of atmospheric conditions.
The project addresses this challenge by providing:

- A structured process for handling weather-related data.
- A machine-learning pipeline for generating estimates from selected features.
- A dashboard for presenting atmospheric information in one interface.
- Short-term outlook panels for viewing model outputs across selected time windows.
- An interactive fuzzy simulator for exploring hypothetical input changes.
- A foundation for future evaluation against independent meteorological observations.

1.1.2 Project Overview

VajraSense consists of several logical components:

1. Data input: Weather-related observations or archived weather data are supplied to the system.
2. Preprocessing: The data are checked, organized, and transformed.
3. Feature engineering: Relevant input variables are prepared for inference.
4. ML-based estimation: The trained classifier generates an estimate for its defined target.
5. Fuzzy inference: The simulator evaluates hypothetical input combinations using configured rules.
6. Backend processing: The application serves the required outputs to the interface.
7. Dashboard visualization: Users view estimates, telemetry, maps, and simulator results.

The outputs depend on the quality, availability, and nature of the input data. Consequently, the system's predictions should be interpreted within the limitations of the current prototype.

1.2 Background

Thunderstorms are associated with convective atmospheric processes. When air near the surface becomes sufficiently buoyant and lifting conditions are favourable, it may rise and form convective clouds. The development of thunderstorms depends on several interacting factors, including atmospheric moisture, instability, lifting mechanisms, and wind conditions.
Several meteorological variables can help describe these conditions.

- Temperature: Influences atmospheric density and the vertical distribution of heat.
- Relative humidity: Indicates how close the air is to saturation at its current temperature.
- Dew point: Provides information about atmospheric moisture.
- CAPE: Represents the potential energy available to an air parcel due to positive buoyancy under the assumptions of the calculation.
- Lifted Index: Compares the temperature of a lifted air parcel with its environment at a specified level; more negative values can indicate greater instability.
- CIN: Represents the inhibition that may prevent an air parcel from rising freely.
- Wind speed and gusts: Describe near-surface airflow and sudden increases in wind.
- Precipitation: Provides information about rainfall-related conditions.
- Cloud cover: Describes the fraction of the sky covered by clouds.

No single variable can reliably confirm that a thunderstorm or lightning event will occur. Atmospheric conditions must be interpreted together, and local storm development also depends on factors that may not be fully represented in a small dataset.

1.2.1 Weather Monitoring and Nowcasting

Nowcasting generally refers to very short-term weather analysis and prediction, often covering the immediate future up to several hours. It may use frequently updated observations, radar, satellite imagery, and numerical weather prediction products.
Radar can provide information about precipitation structures and their movement. Satellite observations

help identify cloud development and atmospheric patterns. Surface measurements and atmospheric model products provide additional information about moisture, stability, pressure, and wind.
VajraSense adopts a prototype-oriented approach by combining selected weather-related variables with model estimation and visualization. Its dashboard presents outputs across configurable short-term windows, but these displayed windows should not be treated as validated forecast skill without suitable independent testing.

1.2.2 Role of Artificial Intelligence and Machine Learning

Machine Learning is a branch of AI in which algorithms learn patterns from data. In supervised learning, a model is trained using input features and associated target labels.
For VajraSense, meteorological variables serve as model inputs. The classifier learns patterns associated with the target defined in the training pipeline. Its output therefore depends on the validity and quality of that target.
A model trained on rule-generated labels may learn to reproduce the rules or their correlations. To establish real-world lightning prediction capability, the model must eventually be evaluated against independent observed lightning events.

1.2.3 Role of Fuzzy Logic

Fuzzy logic is useful when a system represents concepts such as low, moderate, or high using degrees of membership rather than only strict binary categories.
For example, a fuzzy system may assign a CAPE input partial membership in both a “moderate” category and a “high” category. Rules then combine the input memberships to calculate an output.
In VajraSense, the What-If Simulator allows users to adjust selected input values and observe the resulting fuzzy-system output. This helps demonstrate how the configured rules respond to hypothetical conditions. The output is a rule-based score and is not automatically a calibrated probability of lightning.

1.3 Importance of the Project

The importance of VajraSense lies in its attempt to make weather-related data processing and risk estimation easier to explore through a unified interface.

1.3.1 Meteorological Importance

Atmospheric variables provide complementary information about weather conditions. A structured system can make these variables easier to inspect together and may support further investigation into patterns associated with convection.
The project provides a foundation for exploring how selected atmospheric conditions relate to a defined convective-risk target.

1.3.2 Technological Importance

VajraSense demonstrates the integration of several software and data-science concepts:

- Data cleaning and transformation.
- Feature engineering.
- Supervised machine learning.
- Fuzzy inference.
- Backend API development.
- Geographical visualization.
- Interactive user-interface design.

The project therefore demonstrates an end-to-end software pipeline rather than only an isolated model.

1.3.3 Educational Importance

The system can be used as a learning platform for understanding meteorological variables and data-

driven estimation. The What-If Simulator enables users to explore how changing an input affects a fuzzy-rule output.
It also provides an opportunity to study the differences between model predictions, proxy labels, observations, and validated forecasts.

1.3.4 Practical Importance

A future validated version could support weather awareness by combining reliable data sources and presenting concise information to users. Potential applications include educational demonstrations, weather-data exploration, research prototypes, and decision-support experimentation.
However, the present prototype should not be used as a substitute for official weather alerts or emergency instructions.

1.4 Perspective of Stakeholders and Customers

Different stakeholders may benefit from a system that organizes weather-related information and presents it in an accessible format.

Stakeholder	Potential requirement	Possible benefit from VajraSense
Students and researchers	Explore meteorological variables and ML workflows	Educational experimentation and model analysis
Weather enthusiasts	Understand selected atmospheric conditions	Centralized visualization of weather-related information
Data analysts	Inspect input features and model outputs	A structured environment for data analysis
Educational institutions	Demonstrate AI/ML applications	Practical learning and project demonstrations
Disaster-management researchers	Investigate short-term risk-estimation methods	A starting point for further research
Public users	Understand local weather conditions	Potential future benefit after independent validation

1.4.1 User Requirements

The principal user requirements are:

1. The system should present information in an understandable interface.
2. Users should be able to view selected weather-related variables.
3. The dashboard should show model outputs clearly.
4. The interface should support exploration of different time windows where implemented.
5. Users should be able to modify supported simulator inputs.
6. The system should handle missing or invalid inputs appropriately.
7. The interface should distinguish observed data, model estimates, and simulated values.
8. The system should communicate its limitations clearly.

1.4.2 Stakeholder Expectations

Users may expect accurate predictions, timely updates, and clear warnings. Meeting these expectations requires more than a working interface. It also requires trustworthy data, suitable model training, reliable inference, and independent evaluation.
VajraSense addresses the software integration and visualization aspects. Its current validation limitations mean that it should be presented as a prototype rather than a dependable public warning service.

1.5 Objectives and Scope of the Project

1.5.1 Objectives

The primary objective is to develop a prototype that combines meteorological data processing, machine learning, fuzzy logic, and visualization for short-term convective weather-risk estimation.

The specific objectives are:

1. To organize weather-related datasets for the selected study area.
2. To preprocess meteorological variables into a structured format.
3. To derive relevant features from temperature, moisture, precipitation, wind, and atmospheric-stability variables.
4. To implement a machine-learning pipeline for the target defined in the project.
5. To display the system's estimates through a web-based dashboard.
6. To provide a geographical view and atmospheric telemetry.
7. To implement an interactive fuzzy What-If Simulator.
8. To inspect model behaviour using suitable evaluation methods.
9. To identify data and model limitations.
10. To define future improvements required for independent validation.

1.5.2 Scope

The current project scope includes:

- Weather-related data ingestion and processing.
- Feature engineering.
- ML-based estimation using the available training pipeline.
- Fuzzy-logic simulation.
- Dashboard visualization.
- Display of short-term outlook panels.
- Testing of implemented software components.
- Documentation of system limitations and future work.

1.5.3 Limitations of Scope

The current scope does not establish:

- A verified operational lightning forecast.
- A calibrated probability of lightning occurrence.
- An officially approved weather-warning service.
- Independent validation against a complete observed lightning dataset.
- Guaranteed real-time radar or satellite integration.

These limitations are important when interpreting the results.

1.6 Summary

This chapter introduced VajraSense, explained the background of thunderstorm and lightning analysis, and described the importance of combining weather-related data with AI/ML methods. It presented the role of meteorological variables, the potential use of machine learning, and the exploratory function of fuzzy logic. The chapter also identified the project's objectives, stakeholders, scope, and limitations. The next chapter reviews related approaches and explains the problem definition, feasibility, and proposed methodology.

CHAPTER 2: LITERATURE SURVEY & PROPOSED WORK

2.1 Introduction

A literature survey examines existing research, techniques, and technologies related to the problem being addressed. For VajraSense, the relevant areas include meteorological observation, short-term weather prediction, machine learning, atmospheric instability analysis, fuzzy logic, and interactive visualization.

The objective of this chapter is to explain the technical background and identify how the proposed prototype relates to existing approaches. The discussion also identifies a limitation that is central to this project: performance against a constructed proxy target does not establish prediction accuracy against independently observed lightning.

2.2 Literature Survey Table

The following table summarizes relevant methods and documentation. It is a conceptual survey of established approaches; it should not be represented as a set of experimental studies performed by the project team.

Sr. No.	Area / Reference	Method or focus	Relevance to VajraSense	Limitation or consideration
1	World Meteorological Organization — nowcasting and forecasting resources	Short-term weather monitoring and prediction	Establishes the broader meteorological context for nowcasting	Operational nowcasting depends on observations and specialist methods
2	Open-Meteo API documentation	Access to weather and historical weather products	Provides a method for obtaining weather-related variables	Product type, time coverage, and data provenance must be documented
3	ERA5 reanalysis documentation	Historical atmospheric reanalysis	Relevant to historical weather analysis	Reanalysis is not identical to direct local observations
4	Global Forecast System (GFS) documentation	Numerical weather prediction products	Relevant to atmospheric forecast variables	Forecast data can differ in resolution and timing from surface data
5	Scikit-learn documentation	Supervised learning and gradient-boosting classifiers	Supports implementation of the ML pipeline	Performance depends on labels, features, and evaluation design
6	Research on radar-based nowcasting	Uses radar observations to track precipitation and storm structures	Relevant to future integration of measured storm information	Requires reliable radar data and appropriate temporal coverage
7	Research on satellite-based storm monitoring	Uses satellite imagery and derived features	Relevant to future cloud and storm-development analysis	Satellite indicators alone do not guarantee local lightning prediction
8	Research on fuzzy inference systems	Uses membership functions and rules to represent uncertain categories	Relevant to the What-If Simulator	Outputs depend on the chosen rules and membership functions
9	Research on time-	Chronological	Relevant to	Incorrect splitting

	series validation	splitting and leakage prevention	trustworthy model evaluation	can produce overly optimistic results

Note: Add the exact titles, authors, publication years, journal or conference details, and URLs of the research papers you actually consult before final submission. The table above is a thematic survey, not a claim that each row represents a fully reviewed paper.

2.2.1 Meteorological Data Sources

Meteorological systems rely on multiple data sources, including station observations, satellite imagery, radar products, numerical weather prediction, and reanalysis datasets.

These sources differ in spatial resolution, temporal frequency, measurement methods, and coverage. Combining variables from different sources requires attention to timestamps, units, coordinate systems, and data provenance.

VajraSense uses weather-related variables to create an input dataset. The selected variables support exploration of atmospheric conditions, but the absence of independent observed lightning labels limits the conclusions that can be drawn about lightning prediction.

2.2.2 Machine Learning for Weather Analysis

Machine-learning models can identify relationships between input variables and a defined target. Gradient-boosting methods build an ensemble of decision trees in stages, allowing later trees to correct errors made by earlier trees.

The project uses a HistGradientBoostingClassifier in the offline pipeline identified during technical review. This classifier is suitable for structured numerical datasets, subject to appropriate preprocessing and training.

However, a model can only learn the relationships represented in its training data. If labels are generated using thresholds or rules, the model may learn those rules rather than independently learn actual lightning occurrence.

2.2.3 Fuzzy Logic for Risk Estimation

Fuzzy logic can represent categories such as low, medium, and high using overlapping membership functions. A fuzzy inference system applies rules to these memberships and combines the results into an output.

In VajraSense, fuzzy logic is used in the interactive simulator. It allows users to explore how selected inputs influence the configured fuzzy output. The simulator complements the ML component by offering an explicit rule-based method for experimentation.

The two components should not be confused: an ML model and a fuzzy system may produce different outputs because they use different methods, inputs, and assumptions.

2.2.4 Research Gap

The project addresses an implementation gap by combining selected weather-related variables, an ML pipeline, a fuzzy simulator, and an interactive dashboard in one prototype.

The main remaining research gap is independent validation. Reliable lightning observations, suitable time alignment, and rigorous chronological evaluation are required to establish whether the model provides meaningful skill for actual lightning prediction.

2.3 Problem Definition

The problem addressed by VajraSense is the difficulty of bringing multiple weather-related variables together in a structured and understandable short-term convective-risk analysis interface.

Meteorological data may contain temperature, humidity, precipitation, pressure, wind, cloud cover, and instability indicators. Analysing these variables separately may not provide an integrated view of the conditions represented in the dataset.

The project aims to process selected inputs, generate an ML-based estimate for a defined target, and display the output through an interactive dashboard. It also provides a fuzzy-logic simulator for exploring hypothetical input changes.

2.3.1 Formal Problem Statement

To develop an experimental system that accepts structured weather-related input features \(X\), applies a

trained machine-learning model f, and produces an estimate $\hat{y}$ for a predefined target:
$$\hat{y}=f(X)$$
The system then presents the estimate alongside relevant atmospheric variables and visual elements. A separate fuzzy inference component evaluates user-selected hypothetical inputs.
The quality of the estimate depends on the input data, target definition, model training, and evaluation method. Establishing real-world lightning prediction skill requires independent observed lightning labels.

2.3.2 Challenges

The principal challenges include:

- Handling missing or inconsistent weather data.
- Aligning timestamps across different data sources.
- Selecting relevant features without introducing future information.
- Defining a meaningful target for model training.
- Avoiding data leakage during evaluation.
- Ensuring consistency between the offline model and dashboard inference.
- Explaining the meaning of displayed risk scores.
- Distinguishing simulated values from actual observations.
- Obtaining suitable independent lightning observations.

2.4 Feasibility Study

A feasibility study evaluates whether a project can be implemented with the available technology, time, resources, and data.

2.4.1 Technical Feasibility

VajraSense uses widely available software technologies, including Python, Pandas, Scikit-learn, Flask, and web-based visualization tools. These technologies can run on a typical development computer without requiring a dedicated GPU for the described CPU-based pipeline.
The system is technically feasible as a prototype. The principal technical challenge is not merely executing the model, but ensuring that data processing, model inputs, and evaluation are consistent and reliable.

2.4.2 Economic Feasibility

The project primarily uses open-source software and data-access tools. This reduces initial software licensing costs.
Potential costs may include cloud hosting, data storage, network access, and access to specialized datasets. Some observational data sources may have registration, licensing, or access restrictions.

Item	Expected cost consideration
Python and data-processing libraries	Generally available without licensing fees
Machine-learning libraries	Generally available without licensing fees
Local development environment	Existing computer can be used
Web dashboard	Can be developed locally; hosting may incur costs
Weather data	Depends on provider and usage terms
Independent lightning observations	Access conditions and possible costs must be checked
Maintenance	Requires future developer time and testing

2.4.3 Operational Feasibility

The dashboard is designed to make selected outputs accessible through a graphical interface. Users can inspect the displayed information without directly interacting with the underlying Python pipeline.
The system's operational usefulness depends on whether its inputs are current, its outputs are understandable, and its limitations are communicated. Until independent validation is completed, the system should be used for demonstration and experimentation rather than safety-critical decisions.

2.4.4 Schedule Feasibility

The project can be divided into manageable stages: data collection, preprocessing, model development, interface development, integration, testing, and documentation.
A modular implementation allows these activities to be developed and tested separately. A detailed schedule is provided in Chapter 3.

2.4.5 Feasibility Summary

Feasibility type	Assessment	Reason
Technical	Feasible as a prototype	Required tools support data processing, ML, and web development
Economic	Relatively low initial cost	Primarily uses open-source technologies
Operational	Feasible for demonstration	Dashboard organizes outputs into an accessible interface
Data	Partially constrained	Independent lightning observations remain necessary
Schedule	Feasible with prioritization	Modular work can be divided into development stages

2.5 Methodology Used

The project follows a data-processing and model-development workflow.

2.5.1 Data Collection

Weather-related data are collected for the selected study region. Variables include temperature, humidity, dew point, precipitation, pressure, cloud cover, wind, CAPE, LI, and CIN.
The previously prepared datasets cover the period from 1 January 2025 to 19 October 2026. The hourly dataset contains 15,768 records, and the daily dataset contains 657 records, according to the earlier dataset summary.
These counts describe the prepared files; they do not establish that the records are independent ground observations or that they include observed lightning events.

2.5.2 Data Preprocessing

Preprocessing includes loading the dataset, checking timestamps, identifying missing values, checking duplicate records, converting data types, and preparing features.
The process must preserve the time sequence. Imputation and feature construction should be fitted or performed in a way that prevents information from future timestamps from influencing earlier predictions.

2.5.3 Feature Engineering

Feature engineering prepares variables that may help characterize atmospheric conditions. Examples include changes in temperature, wind-related values, moisture indicators, and atmospheric instability variables.
Every feature must be defined consistently during training and inference. For example, a temperature difference must use the same time direction and units throughout the pipeline.

2.5.4 Target Construction

The current implementation includes a rule-derived convective-event target based on configured atmospheric thresholds, weather codes, and precipitation/wind conditions.
Because the target is constructed from rules, it represents the rules' definition of an event rather than an independently observed lightning event. This distinction must be maintained in the methodology and evaluation sections.

2.5.5 Model Training

The offline pipeline uses a HistGradientBoostingClassifier. The model is trained on engineered input features and the target defined by the pipeline.
A chronological training and evaluation split is appropriate for this time-series problem. Preprocessing

should be fitted using training data only, and any overlapping future labels should be handled with suitable separation between training and evaluation periods.

2.5.6 Fuzzy Inference

The fuzzy system accepts selected input values, maps them to configured fuzzy membership categories, evaluates rules, and calculates an output through the chosen defuzzification method.

The simulator allows the user to change input values and observe the resulting score. The result depends on the configured rules and is not equivalent to an independently validated probability.

2.5.7 Visualization

The dashboard displays the output through a map, risk cards, atmospheric telemetry, short-term outlook panels, and the What-If Simulator.

Where any radar or lightning layer uses simulated data, it should be clearly labelled as simulated rather than observed.

2.5.8 Evaluation

Evaluation should use suitable classification metrics and compare the model against a baseline. It should also assess the limitations of the target and the effect of temporal leakage.

To demonstrate actual lightning prediction skill, the project requires independent observed lightning records aligned with the input weather data.

2.6 Summary

This chapter reviewed the technical background of meteorological data analysis, machine learning, fuzzy logic, and weather visualization. It defined the project problem and assessed technical, economic, operational, data, and schedule feasibility. The methodology was described from data collection and preprocessing through feature engineering, model training, fuzzy inference, visualization, and evaluation.

The next chapter explains the project planning and scheduling.

CHAPTER 3: ANALYSIS AND PLANNING

3.1 Introduction

Project planning is an important stage in software development because it establishes the activities, resources, sequence, and expected outcomes required to complete the project.
VajraSense combines data engineering, machine learning, fuzzy logic, backend development, dashboard design, and testing. These components must work together to provide a consistent user experience.
The project is divided into smaller work packages so that individual components can be developed and verified before integration. This approach also helps identify dependencies and risks early.

3.2 Project Planning

3.2.1 Planning Objectives

The main objectives of project planning are:

1. To divide the project into manageable tasks.
2. To identify the software tools and data required.
3. To determine the order in which components should be developed.
4. To allocate time for implementation and testing.
5. To identify technical risks.
6. To prepare the report and demonstration.
7. To define the work required for future improvements.

3.2.2 Work Breakdown Structure

The project is divided into the following phases.

Phase	Activities	Expected output
Phase 1: Requirement Analysis	Define problem, objectives, scope and users	Requirement specification
Phase 2: Data Preparation	Collect, inspect and preprocess weather-related data	Structured dataset
Phase 3: Feature Engineering	Create and validate model features	Feature dataset
Phase 4: Model Development	Train and evaluate the classifier	Model artifact and evaluation results
Phase 5: Fuzzy Logic	Configure membership functions and rules	Working What-If Simulator
Phase 6: Backend Development	Connect model and application components	Backend services
Phase 7: Dashboard Development	Build the map, cards, telemetry and controls	Interactive interface
Phase 8: Integration	Connect the data pipeline, backend and UI	Integrated prototype
Phase 9: Testing	Verify data, inference, API and UI behaviour	Test results
Phase 10: Documentation	Prepare diagrams, report and presentation	Final submission materials

3.2.3 Resource Planning

The project requires the following resources.
Hardware resources

- A computer capable of running Python and a web browser.
- Sufficient storage for datasets, source code, and model files.
- Internet access for obtaining weather-related data and documentation.

Software resources

- Python for data processing and machine learning.
- Pandas for dataset manipulation.
- Scikit-learn for model development.
- Flask for backend services.
- HTML, CSS and JavaScript or the selected frontend stack for the dashboard.
- Leaflet for geographical visualization.
- Git and GitHub for source-code management.
- VS Code or another suitable development environment.

Data resources

- Historical or archived weather-related datasets.
- Atmospheric variables needed for feature engineering.
- Independent lightning observations for future validation.
- Radar and satellite observations where access and licensing permit.

3.2.4 Risk Planning

Risk	Possible effect	Mitigation
Missing or inconsistent data	Incorrect or incomplete model inputs	Validate records and document missing values
Data-source mismatch	Misleading relationships between variables	Record source, units, timestamps and spatial resolution
Target-label limitations	Misleading interpretation of accuracy	Clearly identify proxy labels and obtain independent observations
Data leakage	Overly optimistic evaluation	Use chronological splits and fit preprocessing on training data only
Model mismatch	Different results between training and dashboard	Reconcile the model artifact and feature schema
API failure	Dashboard cannot retrieve outputs	Add error handling and input validation
Simulated data confusion	Users may mistake demonstrations for real observations	Label simulated layers and values explicitly
Limited development time	Incomplete testing or documentation	Prioritize core functions and maintain a task checklist
Deployment configuration errors	Application fails outside the development environment	Test dependencies, paths, configuration and startup instructions

3.2.5 Expected Deliverables

The expected project deliverables are:

1. Weather-data preparation pipeline.
2. Engineered feature dataset.
3. Machine-learning model and inference pipeline.
4. Fuzzy-logic simulator.
5. Backend application.
6. Interactive dashboard.
7. System architecture and design diagrams.
8. Testing and evaluation documentation.
9. Capstone report and presentation.

The status of each deliverable should be confirmed against the final source code before submission.

3.3 Scheduling

A project schedule assigns an approximate duration to each phase. The following is a suggested planning table; replace the durations with the actual dates and activities completed by the team.

Sr. No.	Task	Suggested duration	Dependency
1	Requirement analysis	2 days	None
2	Literature survey	3 days	Requirement analysis
3	Data collection and inspection	3 days	Requirement analysis
4	Data preprocessing	3 days	Data collection
5	Feature engineering	2 days	Preprocessing
6	Model development and evaluation	4 days	Feature engineering
7	Fuzzy-system development	3 days	Input selection
8	Backend implementation	3 days	Model interface
9	Dashboard implementation	5 days	UI and API requirements
10	Integration	3 days	Backend and dashboard
11	Testing and debugging	4 days	Integration
12	Report and presentation	3 days	Implementation and test review

The activities can overlap when the team has sufficient resources. For example, the literature survey and initial dashboard design may proceed while data preparation is underway.

3.3.1 Scheduling Considerations

The following dependencies are important:

- Model development requires prepared data and a clearly defined target.
- Dashboard integration requires a stable interface to the backend.
- Model evaluation requires a defined split strategy and suitable metrics.
- Final documentation should reflect the actual implementation rather than planned features.
- Testing should be repeated after important changes to the data pipeline or model.

3.3.2 Progress Monitoring

Project progress can be tracked using a simple status table.

Task	Status	Completion evidence
Requirements and scope	To be confirmed	Requirement document
Dataset preparation	To be confirmed	Validated data files
Feature engineering	To be confirmed	Feature-generation output
ML pipeline	To be confirmed	Saved model and inference test
Fuzzy simulator	To be confirmed	Working simulator
Dashboard	To be confirmed	Application screenshots
Integration	To be confirmed	End-to-end demonstration
Testing	To be confirmed	Test execution results
Documentation	To be confirmed	Final report

Do not mark a task as completed unless the corresponding output has been verified.

3.4 Summary

This chapter described the planning approach for VajraSense, including the work breakdown structure, resources, risks, deliverables, and suggested schedule. Dividing the system into modules helps reduce implementation complexity and makes testing easier.

The next chapter presents the system design and implementation, including the DFD, block diagram, flowchart, UML diagrams, and graphical interface.

CHAPTER 4: DESIGN AND IMPLEMENTATION

4.1 Data Flow Diagram (DFD)

A Data Flow Diagram represents how data move through a system. It identifies the main inputs, processing components, outputs, and external entities.

For VajraSense, the primary input is weather-related data. The processing stage prepares the data and generates the features needed by the model. The ML component generates an estimate, while the fuzzy component processes hypothetical inputs separately. The dashboard presents the resulting information.

4.1.1 Level 0 DFD — Context Diagram

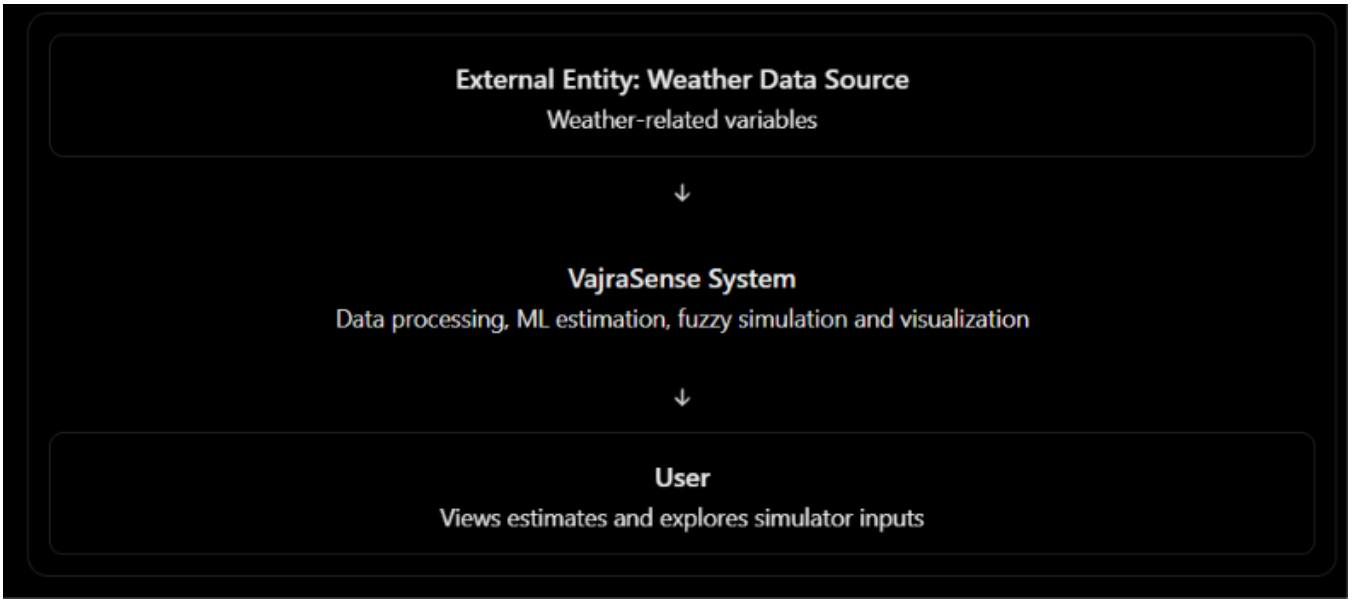


Figure 2. Level 0 Data Flow Diagram of VajraSense.

4.1.2 Level 1 DFD

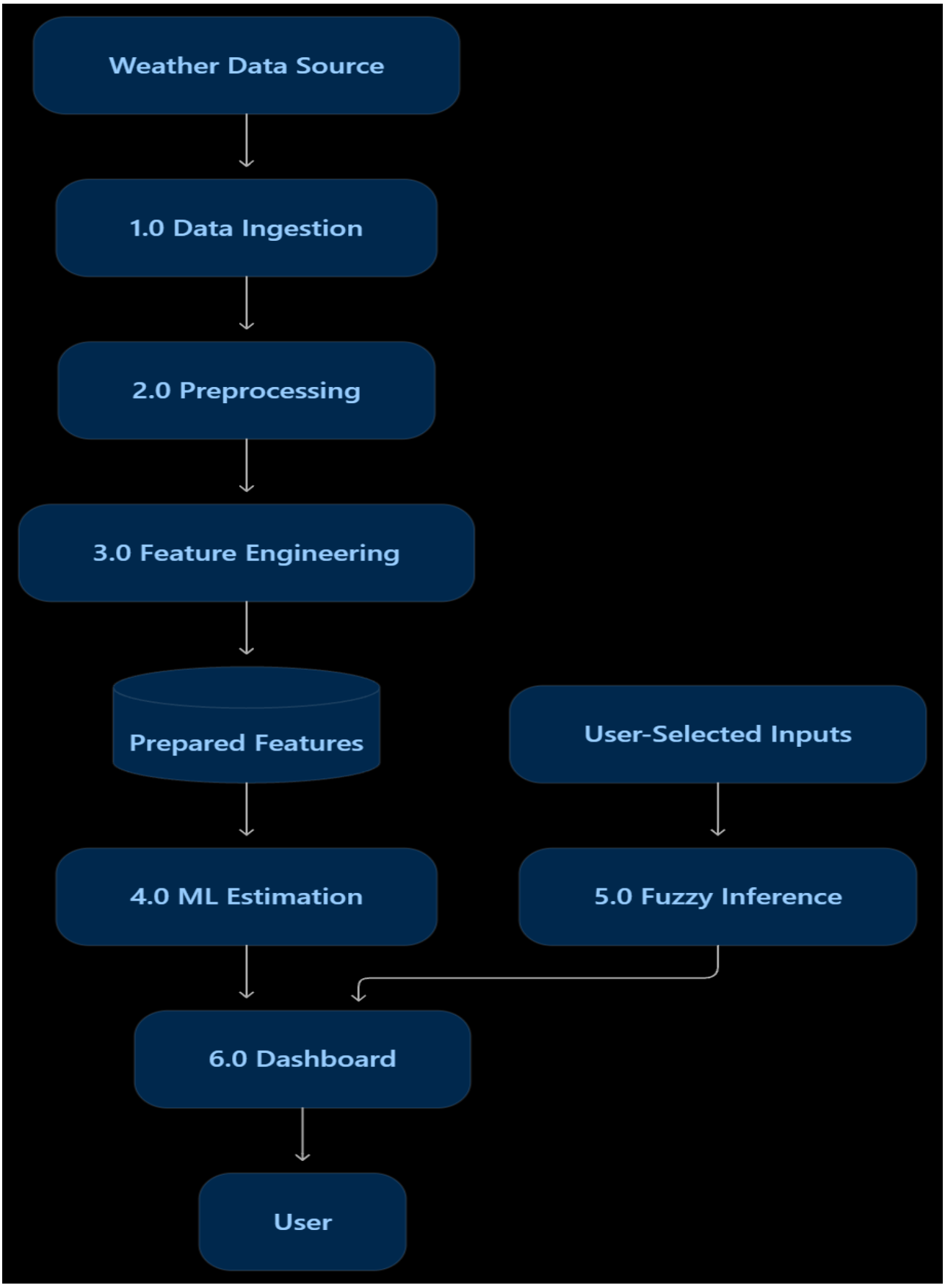


Figure 3. Level 1 DFD of the proposed system.

Explanation of processes

- Data ingestion: Loads weather-related records from the configured source.
- Preprocessing: Checks and transforms the input records.
- Feature engineering: Creates the model input variables.
- ML estimation: Applies the trained classifier to the supported features.
- Fuzzy inference: Processes the hypothetical values supplied through the simulator.
- Dashboard: Presents the outputs in a graphical format.

The exact diagram should be adjusted to match the implemented data flow. For example, if the application loads a saved feature file rather than performing live ingestion, show that file as the actual input.

4.2.1 Explanation of the Block Diagram

Weather-related data: Provides the input variables used for analysis.
Data preprocessing: Organizes records, checks data quality, and prepares the values for feature generation.
Feature engineering: Produces the variables expected by the model.
Trained ML model: Applies the trained classifier to generate an estimate for its defined target.
Fuzzy logic engine: Applies the configured fuzzy rules to user-selected hypothetical inputs.
Backend/API: Makes the relevant outputs available to the dashboard.
Interactive dashboard: Displays the map, risk cards, telemetry, outlook panels, and simulator output.
The ML and fuzzy components are separate analytical paths. Their scores should not be treated as interchangeable unless a defined and validated combination method is implemented.

4.3 Block Diagram

A block diagram illustrates the major components of the system and the relationships between them.

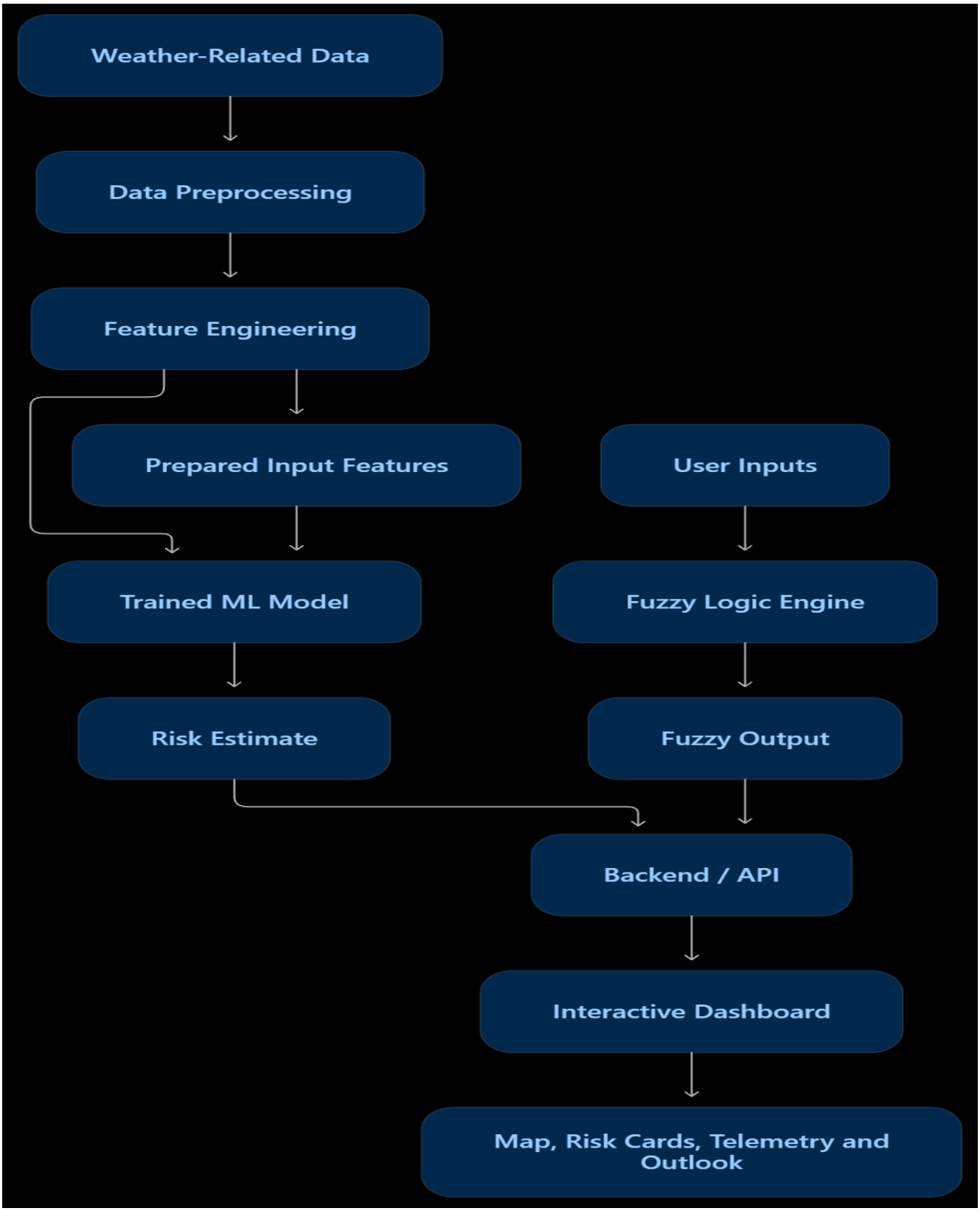


Figure 4. Block diagram of the VajraSense system.

4.3 Flowchart

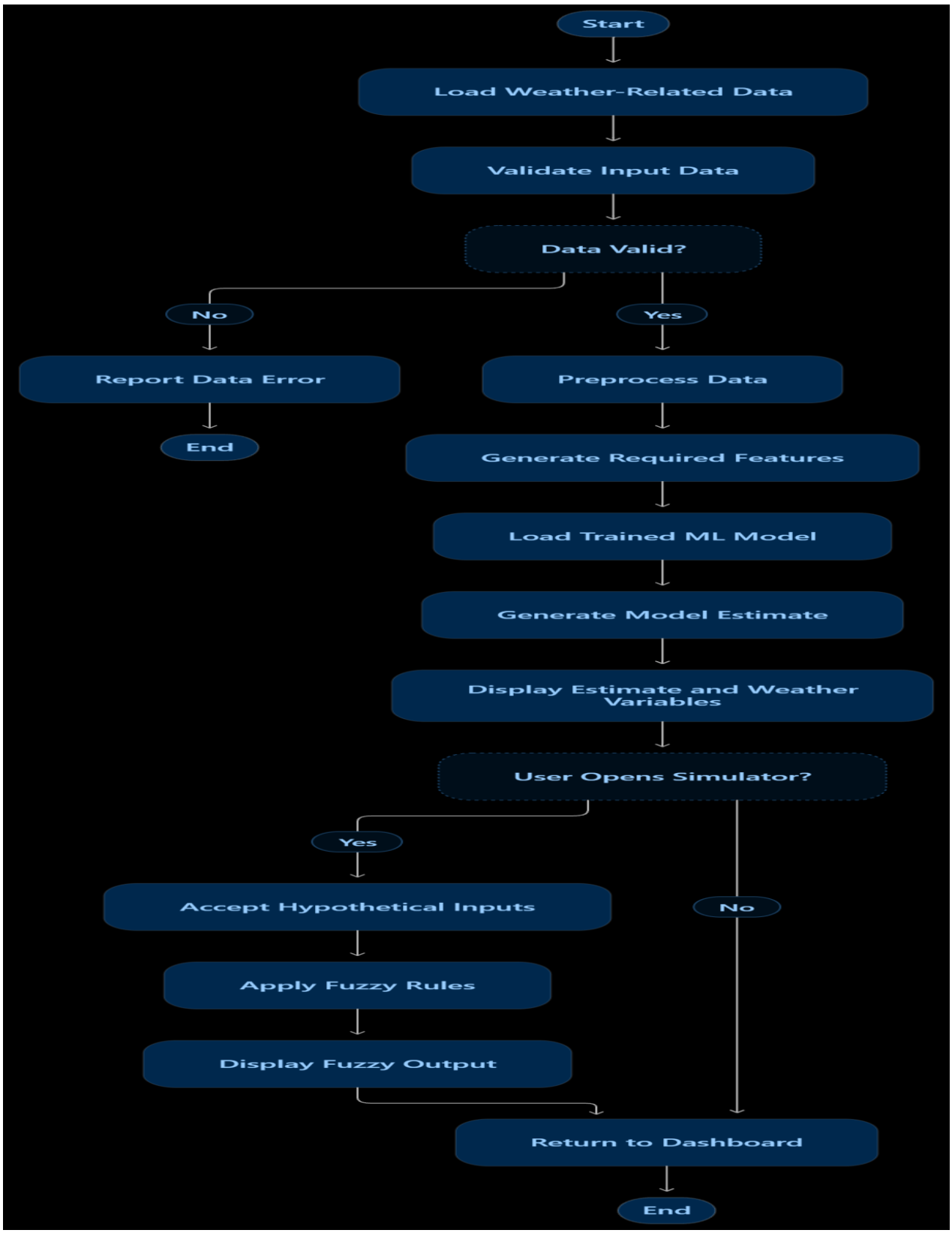


Figure 5. Flowchart of the VajraSense processing workflow.

4.3.1 Flowchart Explanation

1. The system starts and loads the configured weather-related data.
2. The input is checked for missing or invalid values.
3. If the data are invalid, the system should report an error instead of producing an unsupported result.
4. Valid data are preprocessed.
5. The required features are generated.
6. The trained ML model is loaded and used to produce an estimate.
7. The dashboard displays the estimate and relevant variables.
8. If the user opens the simulator, hypothetical inputs are accepted and evaluated using the fuzzy rules.
9. The fuzzy output is displayed separately from the ML estimate.
10. The user can return to the dashboard.

4.4 UML Diagrams

Unified Modeling Language (UML) diagrams describe the structure and behaviour of software systems. The following diagrams provide a conceptual representation of VajraSense.

4.4.1 Use Case Diagram

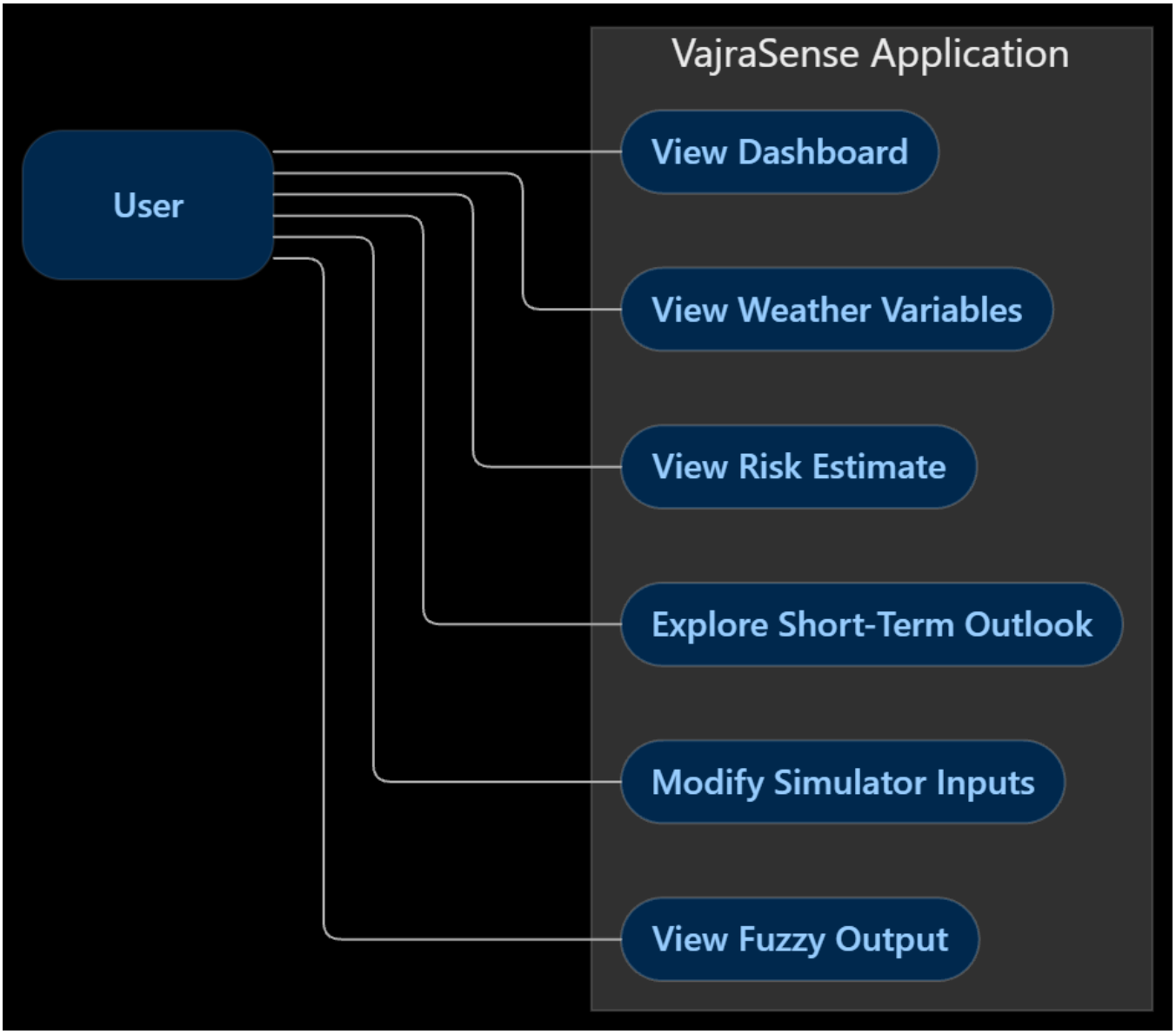


Figure 6. User interaction diagram for VajraSense.

Explanation: The user can view the dashboard, inspect weather variables, examine the system's risk estimate, explore the displayed outlook, and use the fuzzy simulator. Administrative functions such as user login should only be included if they are actually implemented.

4.4.2 Activity Diagram

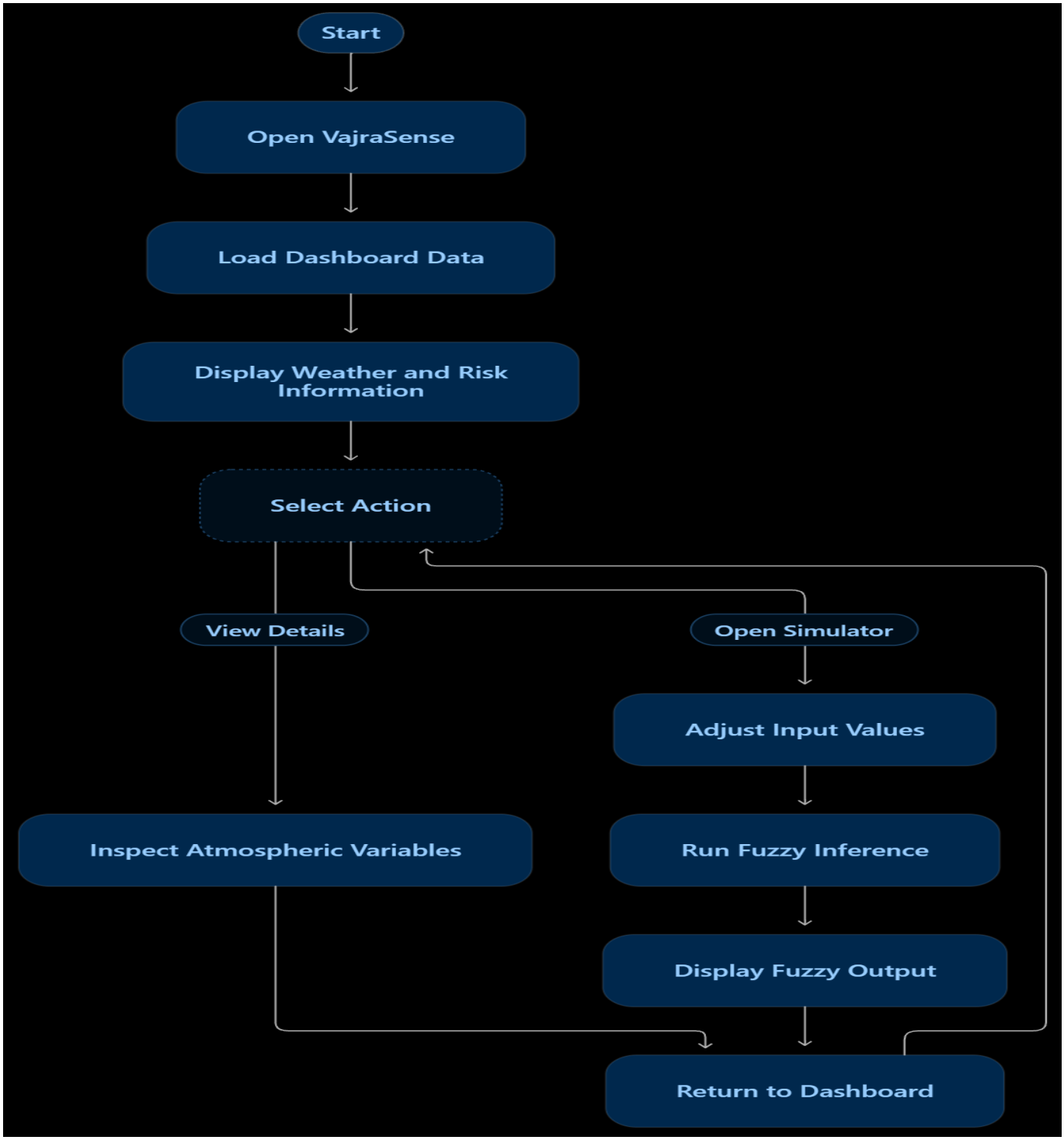


Figure 7. Activity diagram showing user interaction.

4.4.3 Component Diagram

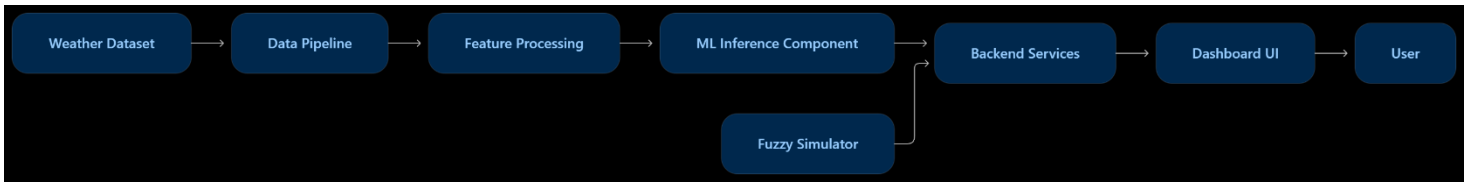


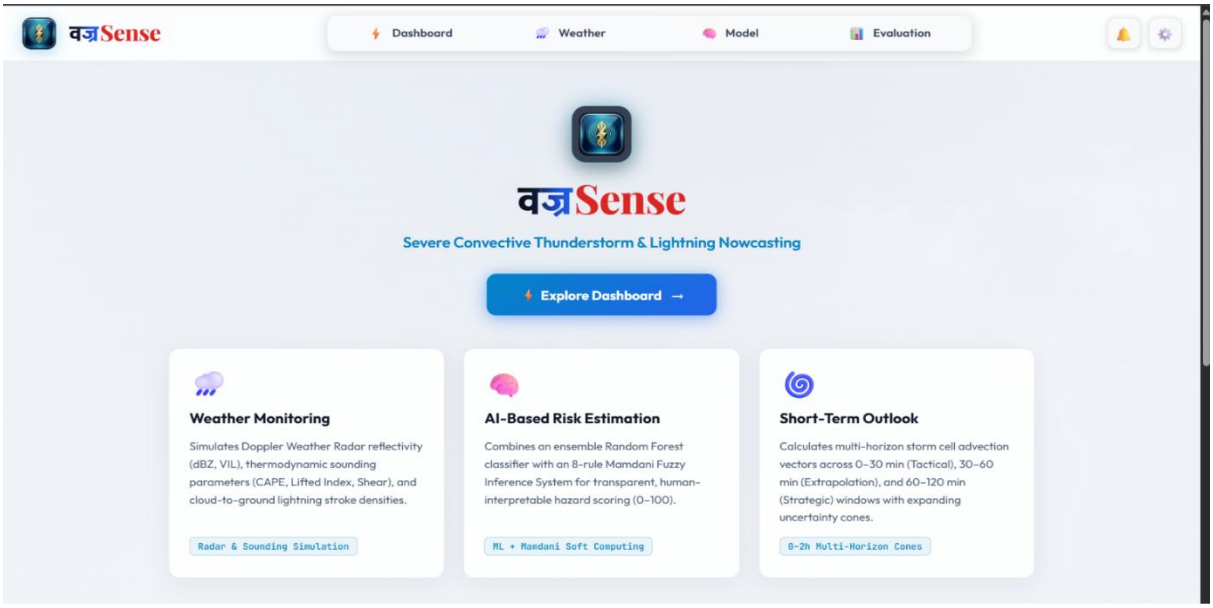
Figure 8. Component-level view of VajraSense.

Explanation: The data pipeline prepares weather inputs, the ML component generates estimates, the fuzzy simulator processes hypothetical values, and the backend makes outputs available to the user interface.
These are conceptual diagrams expressed as flowcharts for ease of preparation. If your college specifically requires standard UML notation, redraw the use case and component diagrams using proper UML shapes in draw.io or another diagramming tool.

4.5 GUI Screenshots

This section documents the actual interface of VajraSense. Insert the screenshots in the order below and keep the captions immediately beneath each image.

4.5.1 Landing Page*Figure 9. VajraSense landing page.*



The landing page introduces the VajraSense project and communicates its focus on thunderstorm and lightning-related weather-risk estimation. It provides a starting point for users to access the dashboard and understand the purpose of the application.
The page establishes the visual identity of the system and helps users navigate to the main functionality.

4.5.2 Main Dashboard

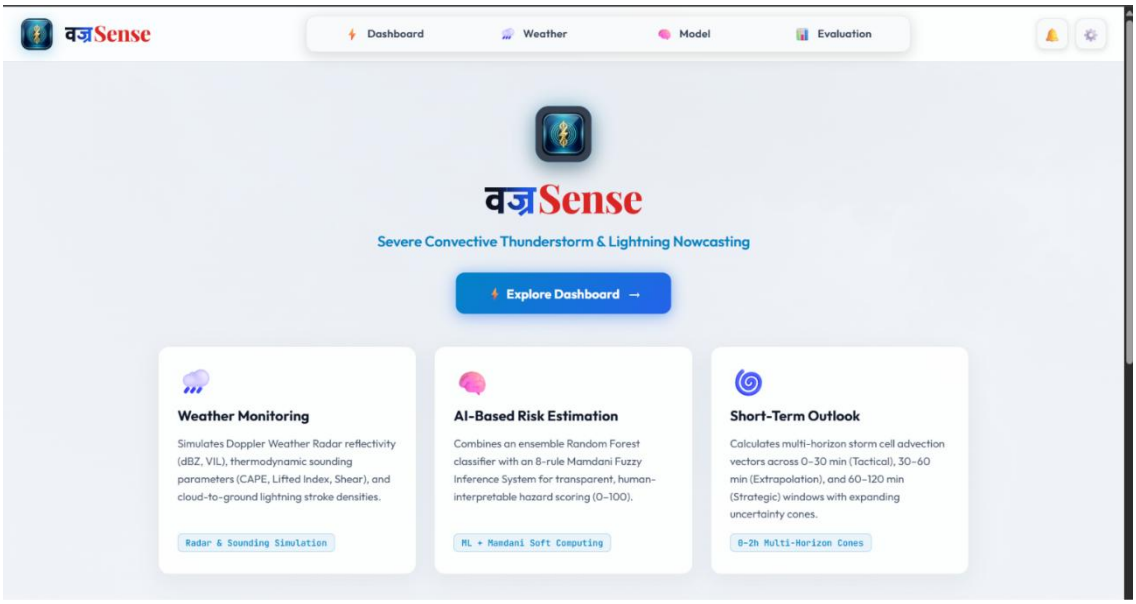


Figure 10. Main VajraSense dashboard.

The main dashboard combines geographical context with risk-related cards and atmospheric telemetry. It is intended to help users inspect the current system state without manually examining separate datasets. The map provides a geographical view, while the information cards summarize the model output. The telemetry area presents selected weather-related variables, and the outlook section displays estimates across configured time windows.
The values shown on the dashboard should be interpreted according to their data source and target definition. Any simulated layer should be labelled accordingly.

4.5.3 Risk Estimation and Explainability

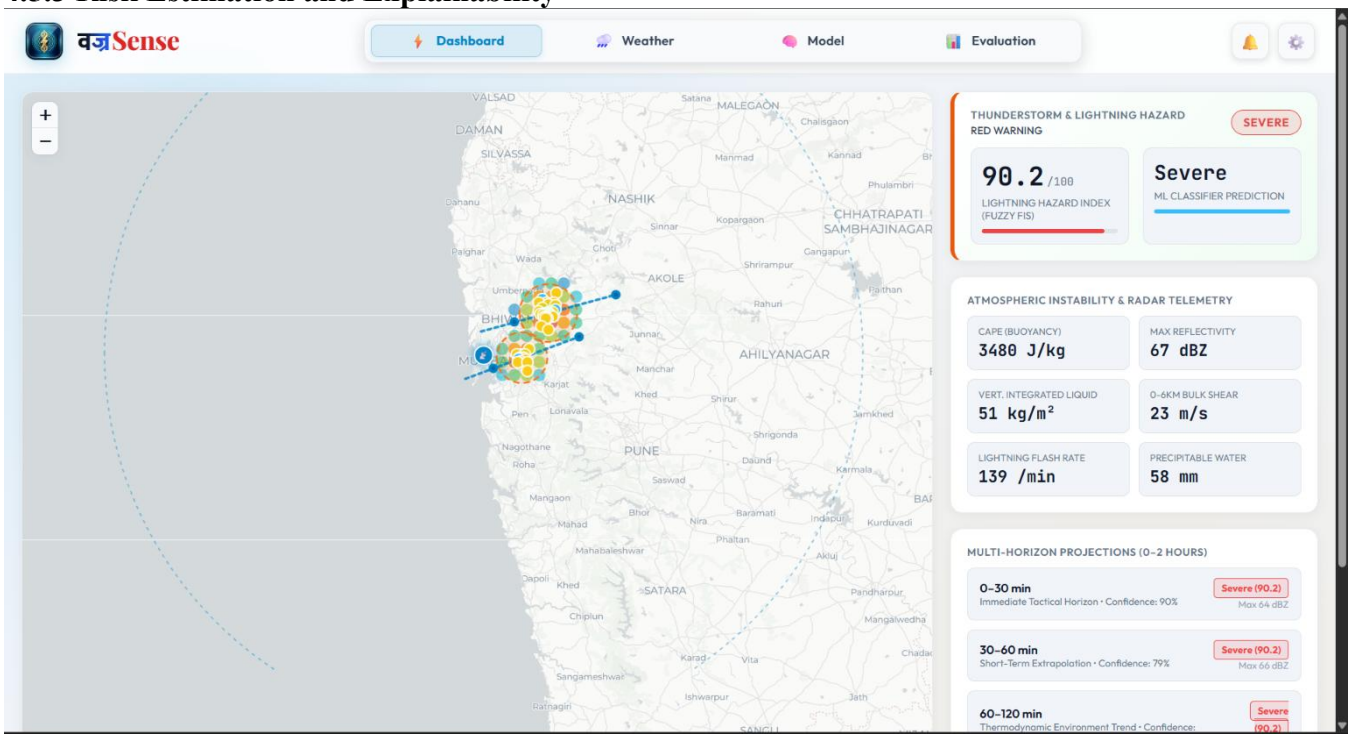


Figure 11. Dashboard showing an alternative risk state and explainability information.

This interface state demonstrates how the dashboard presents a risk estimate and supporting information about the model or configured rules. The explainability trace can help users inspect which inputs or rules are associated with the displayed result.
Such a trace supports transparency, but it does not independently establish causal relationships or prove that the estimate is meteorologically accurate.

4.5.4 Fuzzy What-If Simulator

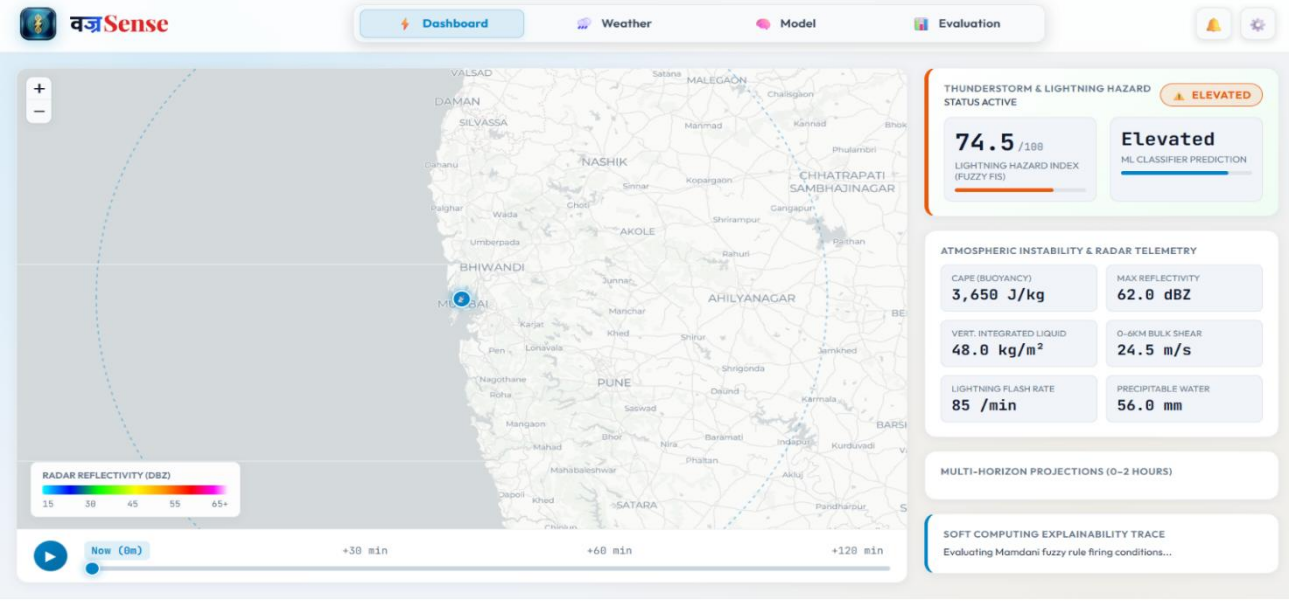


Figure 12. Fuzzy What-If Simulator.

The What-If Simulator allows users to adjust selected inputs, such as CAPE, radar echo intensity, flashrate, and bulk wind shear, where these controls are available in the implementation.
The simulator processes the selected values using the configured fuzzy membership functions and rules. It then displays the corresponding output, allowing users to compare hypothetical scenarios.
This feature is useful for demonstrating the behaviour of the fuzzy system. If a radar or lightning input is simulated, it must not be presented as a real-time measurement.

4.5.5 Interface Summary

The graphical interface combines the major user-facing functions of the prototype:

- Main dashboard.
- Weather-related telemetry.
- Risk-estimation display.
- Short-term outlook panels.
- Explainability information.
- Fuzzy What-If Simulator.

The screenshots provide evidence of the interface design. They should not be used alone as evidence of predictive accuracy.

4.6 Summary

This chapter described the design and implementation structure of VajraSense. The DFD explains the movement of data, the block diagram shows the major system components, and the flowchart describes

the processing sequence. UML-style diagrams represent user interaction and component relationships. The GUI section documents the landing page, dashboard, risk display, and fuzzy simulator. The next chapter discusses the results, future scope, testing, and deployment considerations.

CHAPTER 5: RESULTS AND DISCUSSION

5.1 Actual Results

5.1.1 Overview of Implemented Results

VajraSense demonstrates the integration of weather-related data processing, machine-learning-based estimation, fuzzy logic, and an interactive dashboard. The prototype is organized into separate components so that the data pipeline, model, simulator, backend, and user interface can be developed and tested individually.

The main implementation outcomes are summarized below.

Sr. No.	Component	Result / functionality
1	Weather data	Prepared hourly and daily weather-related datasets
2	Data processing	Data loading, preprocessing, and feature-generation workflow
3	ML pipeline	Classifier for the target defined by the training pipeline
4	Fuzzy logic	Interactive What-If simulation using configured rules
5	Backend	Application services connecting analytical components to the UI
6	Dashboard	Visual display of risk estimates and atmospheric telemetry
7	Map interface	Geographical presentation within the dashboard
8	Short-term outlook	Display of configured prediction horizons
9	Explainability	Supporting information about selected model outputs or rules
10	Documentation	Architecture, interface, limitations, and future improvements

Note: Confirm each component against the final version of the source code before marking it as fully implemented.

5.1.2 Dataset Summary

The previously prepared dataset files were reported to contain the following records.

Dataset	Reported records	Reported coverage
Hourly weather dataset	15,768	1 January 2025 – 19 October 2026
Daily weather dataset	657	1 January 2025 – 19 October 2026

These figures describe the prepared files. The dataset includes weather-related variables, but it should not be described as an independently observed lightning dataset.

5.1.3 Machine Learning Results

The previously reviewed offline pipeline uses a HistGradientBoostingClassifier with engineered features and a rule-derived target.

The earlier technical review reported the following approximate Critical Success Index (CSI) values against that proxy target:

Model / baseline	Previously reported CSI	Interpretation
ML classifier	0.8282	Agreement with the rule-derived target under the reviewed evaluation
Persistence baseline	0.8273	Comparison using the previous state

These are previously reported internal figures, not newly executed test results. Include them in the final submission only after verifying the evaluation code, target definition, data split, and output files.

The difference between these two values is small. Therefore, the results do not establish that the ML model provides a meaningful improvement over persistence. Furthermore, agreement with the rule-derived target does not demonstrate accuracy in predicting independently observed lightning.

5.1.4 Dashboard Results

The dashboard provides a visual interface for examining the model output and associated atmospheric variables.

The main dashboard supports the presentation of:

- Risk-related cards and labels.
- A geographical map.
- Selected atmospheric variables.
- Short-term outlook panels.
- Supporting model or rule information.
- Access to the fuzzy What-If Simulator.

The interface makes it easier to inspect the outputs of the prototype. However, its usability and visual presentation should be evaluated separately from the model's predictive performance.

5.1.5 Fuzzy Simulator Results

The fuzzy simulator allows users to change selected inputs and inspect the corresponding output of the configured fuzzy inference system.

Its main purpose is to demonstrate how the rule system responds to different hypothetical conditions.

The output depends on the configured membership functions, rules, and defuzzification method.

A change in the fuzzy output demonstrates the behaviour of the simulator; it does not independently verify that the same change would occur in a real thunderstorm.

5.1.6 Discussion

The prototype demonstrates the technical feasibility of bringing data preparation, ML inference, fuzzy simulation, and visualization into a single application.

Three aspects require particular attention in future development:

1. Independent validation: The current proxy target does not establish real-world lightning prediction accuracy.
2. Consistent inference: The offline model and dashboard implementation must use the same model artifact and feature definitions.
3. Reliable data: Any future operational version should use well-documented and appropriately aligned observations, including independent lightning labels.

The results should therefore be interpreted as a demonstration of the prototype's architecture and functionality rather than proof of operational forecasting performance.

5.2 Future Scope

VajraSense can be extended in several directions to improve its data quality, predictive capability, usability, and reliability.

5.2.1 Integration of Independent Lightning Data

The most important improvement is the integration of independent observed lightning records. These observations can provide target labels that are separate from the rules used to construct the current proxy target.

The data should be aligned with the weather inputs using suitable timestamps and geographical locations. Data access conditions, resolution, completeness, and licensing must also be documented.

5.2.2 Radar Integration

Radar observations can provide information about precipitation structures and their movement. A future version could integrate suitable radar products to support storm tracking and precipitation analysis. This would require reliable access to the selected radar data, appropriate processing, and a clear distinction between observed and simulated layers.

5.2.3 Satellite Data Integration

Satellite imagery can provide information about cloud development and broader atmospheric conditions. A future version could use suitable satellite products as additional inputs or as a visualization layer. The value of this integration would depend on data availability, spatial and temporal resolution, and the ability to align satellite features with the model's prediction target.

5.2.4 Improved Model Development

Further work could compare different models, such as gradient boosting, random forests, and other appropriate classifiers. The evaluation should use a consistent dataset, target definition, and time-based validation strategy.

Any improvement should be measured against suitable baselines rather than inferred from model complexity alone.

5.2.5 Probability Calibration and Uncertainty

If the system eventually displays probabilities, calibration must be evaluated against independent observations. The dashboard could also communicate uncertainty, data freshness, and missing inputs. A risk score should not be labelled as a probability unless it has an appropriate probabilistic interpretation and supporting validation.

5.2.6 Enhanced Dashboard

Potential interface improvements include:

- Historical risk trends.
- Better data-source indicators.
- Clear labels for simulated inputs.
- Improved visualization of storm movement when suitable observations are available.
- Downloadable reports and evaluation summaries.
- Alerts based on validated criteria, where appropriate.
- Accessible colour schemes and clearer explanations.

5.2.7 Operational Deployment

A future deployment could use a managed server, scheduled data ingestion, model versioning, logging, and monitoring. Before operational use, the system would need reliability testing, security review, independent meteorological validation, and a documented failure-handling process.

5.2.8 Future Scope Summary

The primary future objective is to move from a functional experimental prototype toward a validated, data-driven convective-risk estimation system. Independent observations, sound evaluation methods,

and clear uncertainty communication are more important than adding features without validation.

5.3 Testing

Testing verifies that the individual components behave as expected and that the integrated application processes inputs correctly.

5.3.1 Objectives of Testing

The testing process aims to:

1. Verify that datasets can be loaded correctly.
2. Check the quality of timestamps and input variables.
3. Verify feature generation.
4. Check that the model receives the expected input features.
5. Verify backend responses.
6. Test the behaviour of dashboard components.
7. Check the fuzzy simulator with different inputs.
8. Identify failures and document corrective actions.

5.3.2 Functional Testing

Test ID	Test case	Expected result	Status
T01	Load weather dataset	Dataset loads successfully	To be verified
T02	Check timestamp format	Timestamps are parsed consistently	To be verified
T03	Check missing values	Missing values are identified or handled according to the pipeline	To be verified
T04	Generate model features	Required feature columns are produced	To be verified
T05	Load trained model	Model artifact loads successfully	To be verified
T06	Run model inference	Valid input produces a model output	To be verified
T07	Send backend request	API returns the expected response structure	To be verified
T08	Open dashboard	Main dashboard renders correctly	To be verified
T09	Change simulator inputs	Fuzzy output updates according to configured rules	To be verified
T10	Submit invalid input	System handles the input without an uncontrolled failure	To be verified
T11	Check simulated layers	Simulated information is clearly labelled	To be verified
T12	Run evaluation pipeline	Metrics are generated using the documented split and target	To be verified

Important: The table is a testing plan. Replace “To be verified” with the actual outcome only after executing each test.

5.3.3 Data Validation Testing

Data validation is especially important for weather-related applications. Incorrect timestamps, duplicated rows, missing values, and inconsistent units can affect feature generation and model inference.
The following checks are recommended:

- Confirm that timestamps are in the expected format and timezone.
- Check that records are sorted chronologically.
- Identify duplicate timestamps.
- Inspect missing and invalid values.
- Verify measurement units.
- Confirm that variables from different sources are appropriately aligned.
- Ensure that feature engineering does not use future information.
- Ensure that training and inference use consistent feature definitions.

5.3.4 Model Testing

Model testing should verify both the implementation and the evaluation methodology.
The model should be tested on a held-out chronological period. Any preprocessing transformations should be fitted using the training data only. If labels overlap across forecast horizons, suitable separation between training and evaluation periods should be considered.
The model should also be compared with an appropriate baseline. Evaluation metrics should be calculated from verified predictions and labels, and the report should identify whether the labels are independent observations or rule-derived proxies.

5.3.5 User Interface Testing

The dashboard should be tested for correct display and interaction.
Tests should verify that:

- The main page loads without unexpected errors.
- Risk cards display the intended values.
- Atmospheric telemetry is labelled correctly.
- The map displays the intended region and layer.
- Timeline and outlook controls behave as expected.
- The simulator accepts valid input values.
- Invalid or unavailable data are communicated clearly.
- The interface does not misrepresent simulated data as observations.

5.3.6 Testing Summary

A complete test report should include the test case, input, expected result, actual result, status, and evidence. Screenshots can support UI testing, while logs and evaluation output can support backend and model testing.
A working dashboard alone does not prove that the underlying model has been validated for actual lightning prediction.

5.4 Deployment

Deployment refers to making the application available in an environment where users can access it.

5.4.1 Deployment Architecture

A typical deployment structure for VajraSense consists of:

1. Data layer: Stores the prepared datasets and relevant input files.
2. Model layer: Stores the trained model and associated feature definitions.
3. Backend layer: Processes requests and provides model or simulator outputs.
4. Frontend layer: Displays the dashboard and accepts user interactions.
5. Runtime environment: Provides the required Python packages, configuration, and server process.

The actual deployment diagram should match the final architecture of the project.

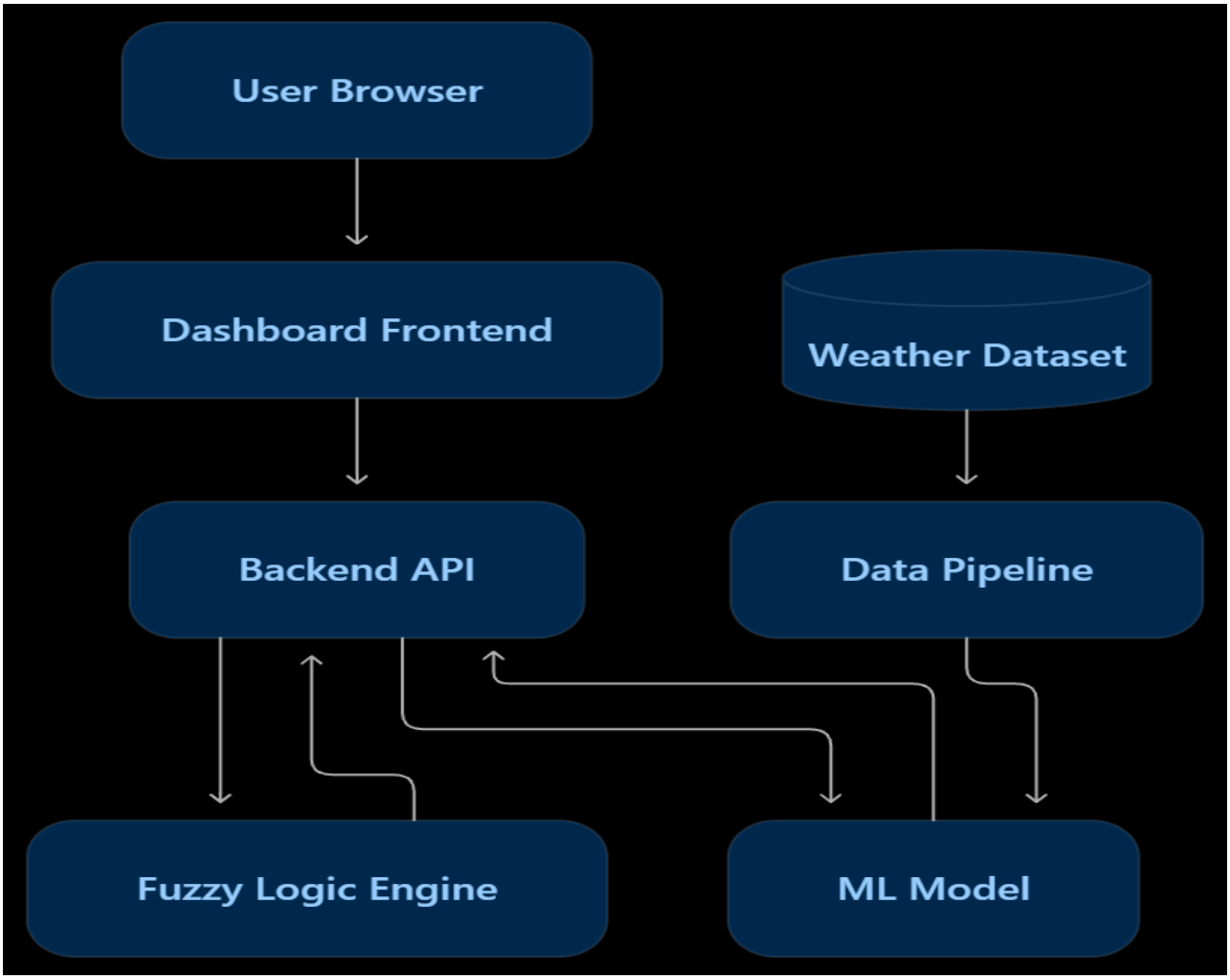


Figure 13. Conceptual deployment architecture of VajraSense.

5.4.2 Local Deployment

For a local demonstration, the application can be run on the development computer after the required dependencies and configuration are prepared.

The general process is:

1. Obtain the project source code.
2. Create and activate a Python virtual environment.
3. Install the dependencies listed in the project's requirements file.
4. Verify the required data files and model artifacts.
5. Configure any required environment variables.
6. Start the backend using the project's documented startup command.
7. Start the frontend if it runs as a separate process.
8. Open the application in a web browser.
9. Verify that the dashboard and simulator work correctly.

Use the actual startup commands documented in your repository rather than copying generic commands into the final deployment section.

5.4.3 Deployment Requirements

Before deployment, verify:

- All required packages are listed and compatible.
- Model and dataset paths are correct.
- The backend and frontend configuration is consistent.

- API errors are handled appropriately.
- No secrets or private credentials are committed to the repository.
- The model artifact is loaded from a trusted source.
- Logs do not expose unnecessary sensitive information.
- Simulated data and model limitations are clearly communicated.

5.4.4 Deployment Limitations

A successful local demonstration does not establish that the system is ready for public or operational use.

A production deployment would require dependable data ingestion, monitoring, security, model version control, failure recovery, and independent meteorological validation. Any use for safety-related decisions would require substantially more rigorous review.

5.5 Summary

This chapter discussed the implementation outcomes, model-evaluation limitations, future scope, testing strategy, and deployment requirements of VajraSense. The prototype demonstrates the integration of several software and data-science components. Further work is required to establish independent predictive performance and operational reliability.

CHAPTER 6: CONCLUSION

6.1 Conclusion

VajraSense is an AI/ML-based prototype designed to explore short-term convective weather-risk estimation using meteorological variables, machine learning, fuzzy logic, and interactive visualization. The project brings together a data-processing pipeline, feature engineering, a machine-learning classifier, a fuzzy What-If Simulator, backend services, and a dashboard. The dashboard provides a unified interface for viewing the system's estimates, inspecting selected atmospheric variables, and exploring hypothetical input conditions.

The project demonstrates the practical integration of Python-based data processing and machine learning with web application development. It also provides an opportunity to study the relationship between input data, target construction, model evaluation, and user-facing predictions.

An important finding is that model evaluation must be interpreted in the context of the target being predicted. The current implementation includes a rule-derived convective-event target, and the available weather-related dataset does not establish independent observed lightning labels. Consequently, previously reported model metrics should not be interpreted as verified lightning-forecast accuracy.

The project therefore represents a foundation for further experimentation rather than a completed operational warning service. Future development should prioritize independent lightning observations, correct temporal validation, consistency between training and inference, and transparent communication of uncertainty.

Overall, VajraSense demonstrates how meteorological data processing, AI/ML, fuzzy inference, and visualization can be integrated into a single experimental platform. With improved data sources and rigorous validation, the architecture could be extended into a more reliable research and decision-support tool.

6.2 Limitations

The principal limitations identified in the project are:

1. Independent lightning labels: The described dataset does not establish an independent set of observed lightning events for model validation.
2. Rule-derived target: The current target depends on predefined rules and may not correspond directly to actual lightning occurrence.
3. Evaluation limitations: High agreement with a proxy target does not establish independent forecasting skill.
4. Temporal leakage risk: Preprocessing, feature construction, and future-label generation require careful time-aware handling.
5. Data-source alignment: Variables obtained from different weather products may have different temporal and spatial characteristics.
6. Model consistency: The offline training pipeline and dashboard inference implementation must use consistent features and model artifacts.
7. Simulated data: Any simulated radar or lightning inputs must be distinguished from measured observations.
8. Operational readiness: The system requires additional validation, monitoring, and failure-handling before operational use.

These limitations provide a clear direction for future research and improvement.

REFERENCES

Use the following references as a starting list. Check the specific documentation versions and publication details you consult, and format them according to your college's required citation style.

1. Open-Meteo, Historical Weather API Documentation. Available at: https://open-meteo.com/en/docs/historical-weather-api

2. Open-Meteo, Historical Forecast API Documentation. Available at: https://open-meteo.com/en/docs/historical-forecast-api

3. Scikit-learn Developers, HistGradientBoostingClassifier Documentation. Available at: https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.HistGradientBoostingClassifier.html

4. Scikit-learn Developers, TimeSeriesSplit Documentation. Available at: https://scikit-learn.org/stable/modules/generated/sklearn.modelection.TimeSeriesSplit.html

5. India Meteorological Department, Mumbai Radar Animation. Available at: https://mausam.imd.gov.in/index_radar_animation.php?id=Mumbai

6. Indian Space Research Organisation, Meteorological and Oceanographic Satellite Data Archival Centre (MOSDAC). Available at: https://www.mosdac.gov.in/

7. Hersbach, H., et al. (2020). "The ERA5 Global Reanalysis." Quarterly Journal of the Royal Meteorological Society, 146, 1999–2049. DOI: https://doi.org/10.1002/qj.3803

8. Pedregosa, F., et al. (2011). "Scikit-learn: Machine Learning in Python." Journal of Machine Learning Research, 12, 2825–2830. Available at: https://www.jmlr.org/papers/v12/pedregosa11a.html

9. ECMWF, ERA5 Reanalysis Documentation. Available at: https://www.ecmwf.int/en/forecasts/dataset/ecmwf-reanalysis-v5

10. National Centers for Environmental Prediction, Global Forecast System (GFS). Available through NOAA/NCEP resources: https://www.nco.ncep.noaa.gov/pmb/products/gfs/

11. Add the exact documentation or research papers consulted for fuzzy logic, lightning observation datasets, and meteorological nowcasting.

12. Add the actual dataset access dates and source URLs used to create the project datas